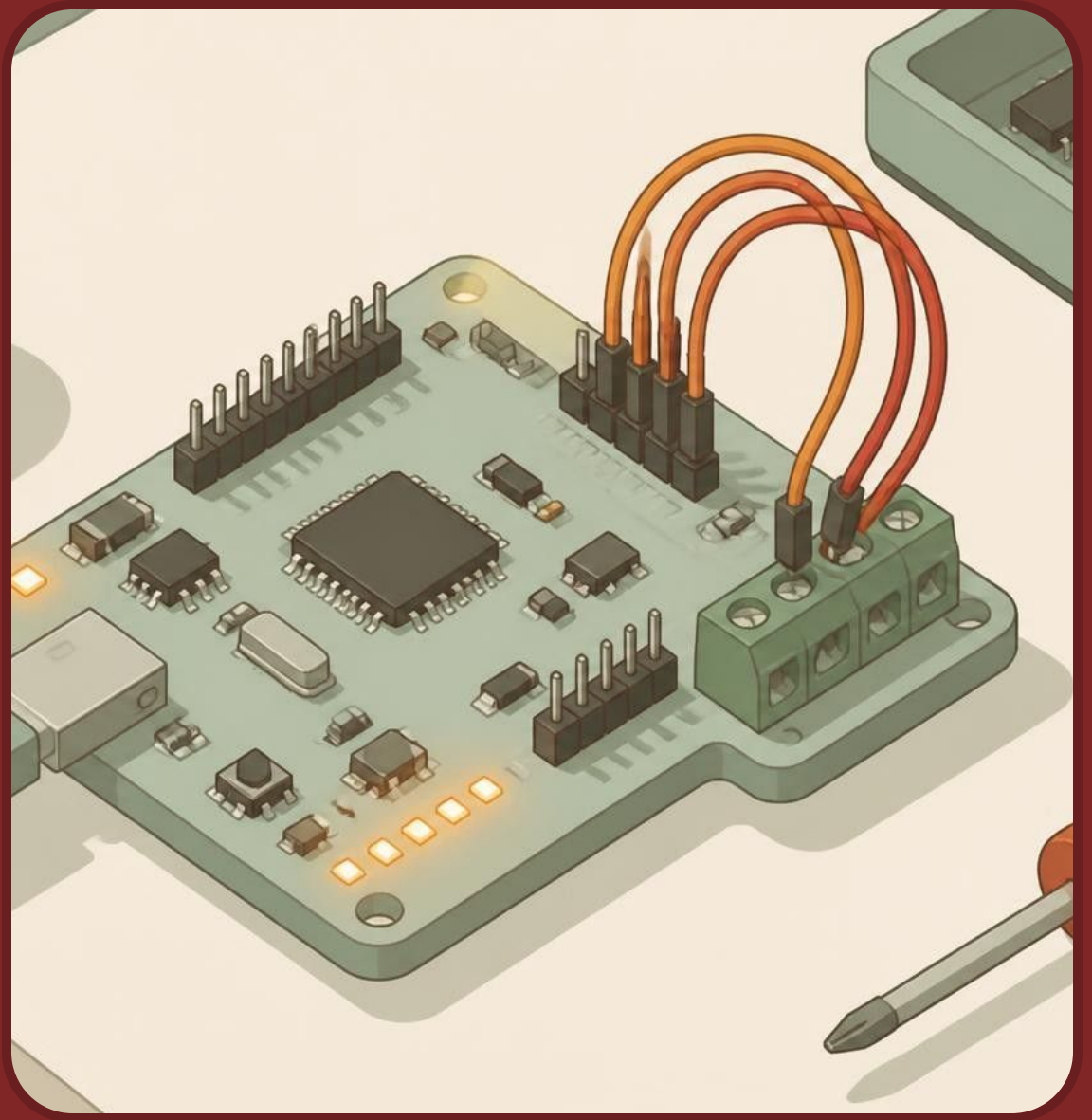
Od szkicu do projektu

Architektura, kod i sprzęt poziom wyżej

Kod zaczyna działać stabilnie, gdy przestaje polegać na delay().

ARDUINO DLA ŚREDNIO ZAAWANSOWANYCH

Przewodnik inżynierski · wzorce mikrokontrolerów



Made with GAMMA

WARSTWY SYSTEMU

Arduino to nie tylko płytka

Połącz trzy warstwy: mikrokontroler, biblioteki i własną logikę. Myśl o projekcie jak o małym systemie, nie pojedynczym eksperymencie.

01

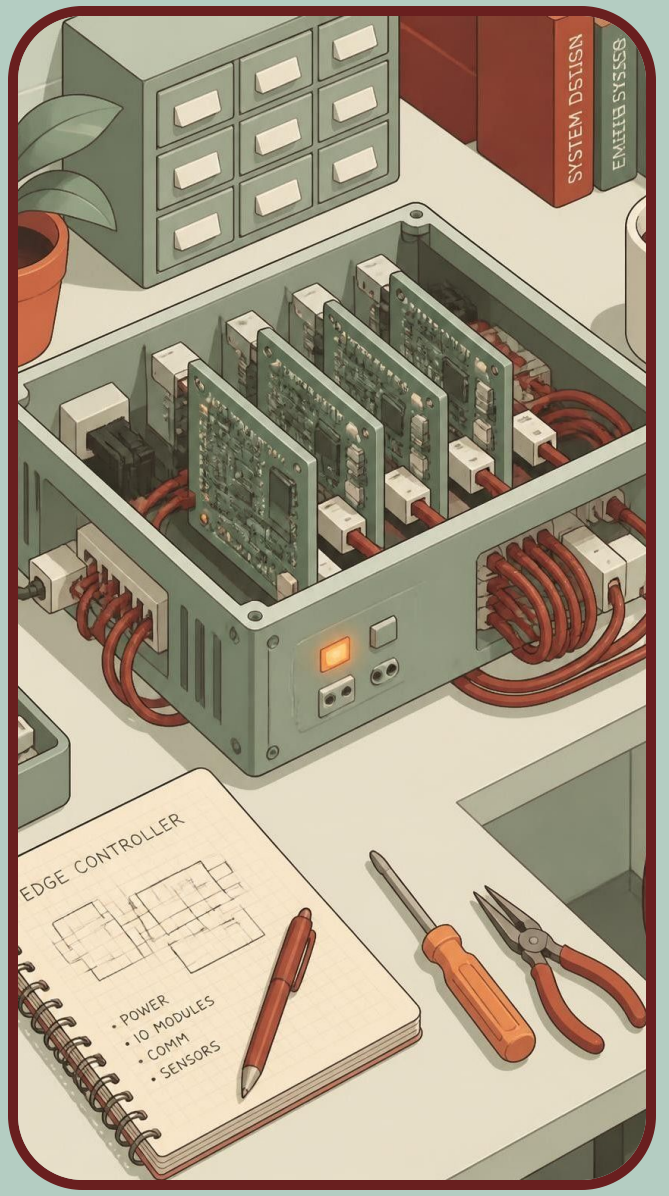
Mikrokontroler

02

Biblioteki

03

Własna logika



Made with GAMMA

CYKL ŻYCIA MIKROKONTROLERA

Co naprawdę wykonuje Arduino?

Brak wielozadaniowego systemu operacyjnego oznacza, że procesor wykonuje instrukcję po instrukcji. To pętla główna definiuje responsywność całego urządzenia.

01

RESET

Wektor startowy i bootloader.

02

setup()

Jednorazowa konfiguracja pinów i magistral.

03

loop()

Szybki obrót pętli bez blokowania.

04

Zdarzenie

Upływ czasu millis() lub zmiana na pinie.

05

Powrót

Aktualizacja stanu i natychmiastowy zwrot.



Jeśli jakikolwiek krok zatrzyma procesor, całe urządzenie traci słuch.

W pętli nie czekamy na zakończenie operacji — sprawdzamy wyłącznie, czy nadszedł czas na kolejny atomowy krok.

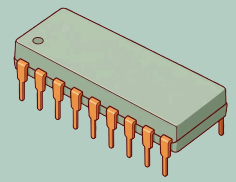
Made with GAMMA

Pin nie jest abstrakcją

Cyfrowe wejście/wyjście, odczyt analogowy, PWM i komunikacja mają różne ograniczenia. Przed kodem sprawdź napięcie, tryb pinu i wspólną masę.

Wywołanie `digitalWrite()` lub `analogRead()` steruje fizycznym tranzystorem wewnątrz krzemu; sprawdź bilans prądowy i poziomy napięć.

Cyfrowe GPIO: wydajność prądowa i pull-up. Typowy pin dostarcza bezpiecznie 20 mA (maks. 40 mA na ATmega328P). Wejście bez rezystora podciągającego działa jak antena zbierająca zakłócenia otoczenia.



PWM to nie jest prawdziwe napięcie analogowe. To szybkie kluczowanie stanów 0 V i 5 V ze zmiennym wypełnieniem. Odbiornik indukcyjny lub silnik wymaga diody zwrotnej zabezpieczającej układ przed przepięciami.

Wspólna masa (GND) to warunek konieczny. Sygnał logiczny to różnica potencjałów. Bez połączonej masy między zasilaczem, modułem a Arduino odczyty magistral i przetworników będą losowe.

Dokumentacja płytki, w tym nota katalogowa układu (datasheet), jest integralną częścią specyfikacji programu.

Made with GAMMA

ARCHITEKTURA CZASU

Od blokowania do reaktywnej pętli

Przejście z delay() na pomiar przez millis() pozwala obsługiwać niezależne zadania bez blokowania reszty logiki.

BLOKUJĄCE DELAY()

Jedno zadanie blokuje świat

delay() zatrzymuje całą logikę: płytka czeka, a naciśnięcie przycisku lub pakiet z portu szeregowego zostaje pominięty. digitalWrite(LED_PIN, HIGH); → delay(1000);

REAKTYWNE MILLIS()

Zadania z własnym interwałem

Sprawdzaj znacznik czasu przy każdym obiegu; pętla działa natychmiast, obsługuje równolegle wiele modułów i reaguje na wejścia. if (currentMillis - lastBlink >= 1000) → lastBlink = currentMillis; → toggleLed(); checkInputs();



Wielozadaniowość na mikrokontrolerze bez RTOS to po prostu harmonogram oparty na upływie czasu.

Made with GAMMA

MASZYNA STANÓW (FSM)

Stan programu porządkuje zachowanie

Modeluj tryby jako stany: IDLE, MEASURE, ALERT, ERROR. Przejścia wywołuj zdarzeniami, nie przypadkowymi warunkami rozsianymi po kodzie. Mała maszyna stanów często zastępuje serię zagnieżdżonych if-ów.

STAN	REAKCJA
IDLE	Oczekiwanie na start; uśpione peryferia, dioda czuwa.
MEASURE	Próbkowanie czujnika i uśrednianie wyników w buforze.
ALERT	Przekroczenie progu: syrena, alarm i raport przez Serial.
ERROR	Awaria magistrali lub timeout: bezpieczny reset modułu.



W pętli głównej `switch (currentState)` porządkuje przejścia i reakcje.

WZORZEC KODU

```
enum State { IDLE, MEASURE, ALERT, ERROR };
State state = IDLE;

void loop() {
  switch (state) {
    case IDLE: if (buttonPressed()) state = MEASURE; break;
    case MEASURE: if (readSensor() > THRESHOLD) state = ALERT; break;
    case ALERT: triggerAlarm(); break;
    case ERROR: recoverBus(); break;
  }
}
```

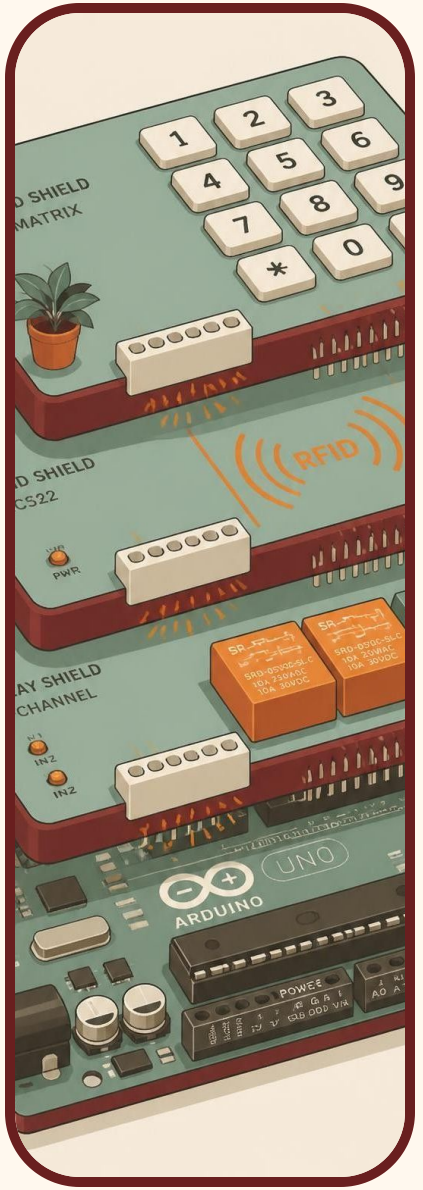
Made with GAMMA

Biblioteka to kontrakt, nie magia

Dołączenie nagłówka włącza cudzy kod do twojej przestrzeni adresowej. Weryfikuj alokację pamięci RAM oraz użycie blokującego delay().

INICJALIZACJA I BŁĘDY	Czy begin() zwraca boolean potwierdzający obecność modułu? Zawsze sprawdzaj kod powrotu.
ZASOBY I TIMERY	Biblioteki silników lub serwomechanizmów blokują konkretne timery sprzętowe, wyłączając PWM na wybranych pinach.
WERSJONOWANIE KODU	Zapisuj dokładne wersje bibliotek w dokumentacji projektu lub korzystaj z menedżera zależności (np. PlatformIO).

`Oficjalny katalog Arduino (WiFi, SD, Servo, Stepper, LiquidCrystal) to sprawdzony punkt wyjścia, ale wymaga zrozumienia ograniczeń pamięci RAM.`



Made with GAMMA

MAGISTRALE DANYCH

Komunikacja: wybierz właściwy kanał

Każdy kanał to kompromis: liczba przewodów, szybkość i adresowanie.

MAGISTRALA	KIEDY UŻYĆ	MOCNA STRONA	TYPOWA PUŁAPKA
UART (Serial)	Debugowanie, PC, modemy GSM/GPS	Tylko 2 linie (TX/RX), uniwersalny terminal	Zajmuje piny programowania USB na płytkach UNO
I2C (Two Wire)	Czujniki, zegary RTC, małe ekrany	Dziesiątki urządzeń na magistrali SDA/SCL	Wymaga rezystorów pull-up; kolizje adresów 7-bit
SPI	Karty SD, wyświetlacze TFT, szybkie ADC	Wysoka prędkość transmisji (rzędu MHz)	Wymaga osobnej linii Chip Select (CS) dla każdego modułu



Wybór magistrali decyduje o topologii płytki drukowanej i zapotrzebowaniu na piny mikrokontrolera.

Made with GAMMA

PRZETWARZANIE SYGNAŁÓW

Odczyt czujnika wymaga decyzji

Surowa liczba z przetwornika ADC (0–1023) rzadko nadaje się do bezpośredniego użycia. Dobry projekt rozdziela akwizycję danych od ich interpretacji i reakcji wykonawczej.

1. Filtracja szumów. Średnia krocząca (moving average) lub filtr dolnoprzepustowy wygładzają skoki napięcia i zakłócenia sieciowe.

2. Kalibracja fizyczna. Przelicz wolty na jednostki fizyczne (°C, hPa, cm) za pomocą wzoru liniowego i dwóch punktów pomiarowych.

3. Histereza progowa. Oddziel próg załączenia od wyłączenia, aby przekaźnik nie drżał na granicy wartości granicznej.



Oddziel surowy odczyt sprzętowy od reguł biznesowych: funkcja pomiaru nie powinna sama włączać silnika.

FILTR DOLNOPRZEPUSTOWY W C++

```
// Prosta filtracja wykładnicza EMA
float filteredValue = 0.0;
const float alpha = 0.15; // Waga nowej próbki

void updateSensor() {
  int raw = analogRead(A0);
  filteredValue = (alpha * raw)
                + ((1.0 - alpha) * filteredValue);
}
```

Kilka linii kodu eliminuje potrzebę stosowania zewnętrznych kondensatorów filtrujących w prostych układach.

Made with GAMMA

METODYKA DIAGNOSTYKI

Debuguj warstwami

01 **Sprawdź zasilanie, masę i połączenia;** zmierz napięcie pod obciążeniem i ciągłość GND.

02 **Wypisz wejścia i ważne przejścia stanu przez Serial.**

03 **Testuj bibliotekę i własną logikę osobno;** zacznij od przykładowego szkicu.

04 **Wytnij 90% kodu;** zostaw najmniejszy powtarzalny przypadek (MRE).



Inżynierskie podejście: debuguj pomiarami, nigdy domysłami.

Made with GAMMA